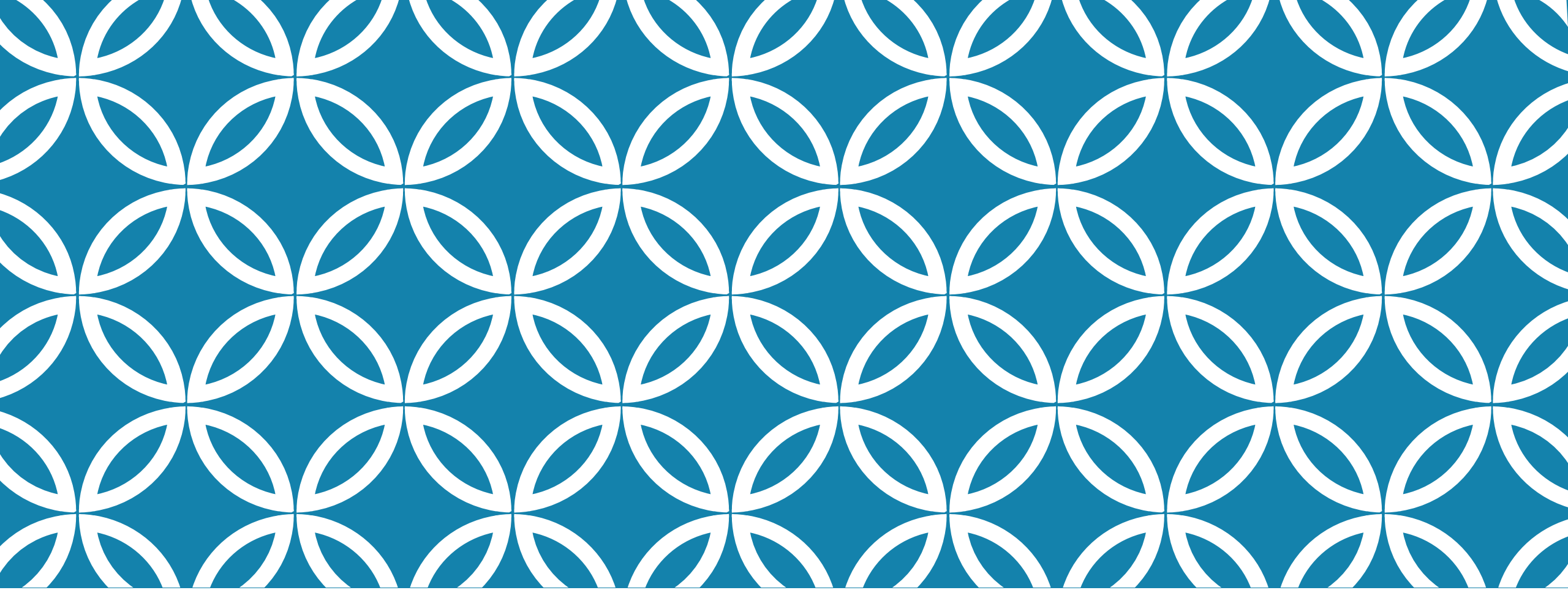
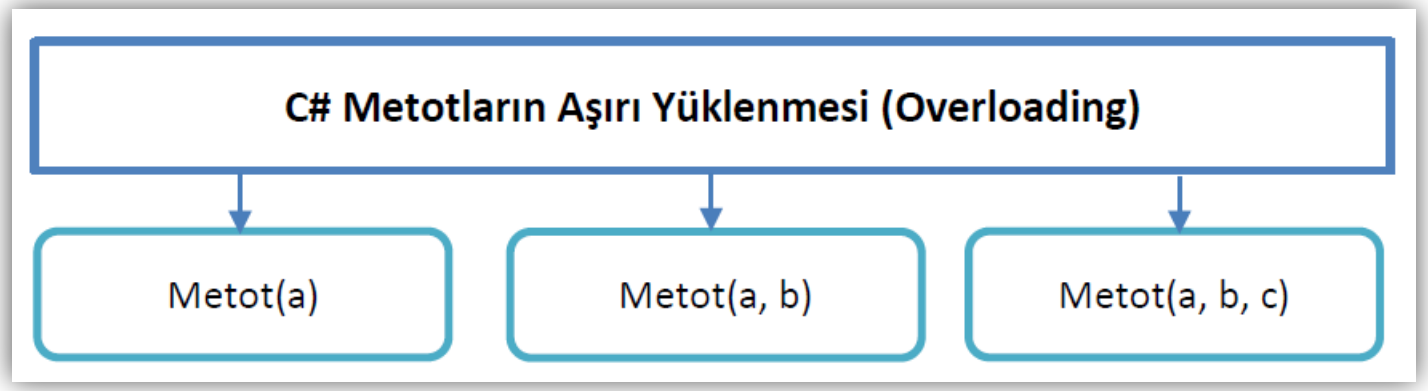


GÖRSEL PROGRAMLAMA-II

Öğr. Gör. Erhan AKAGÜNDÜZ

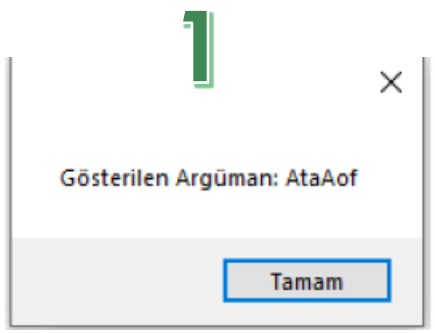
METOT AŞIRI YÜKLEME (OVERLOADING)

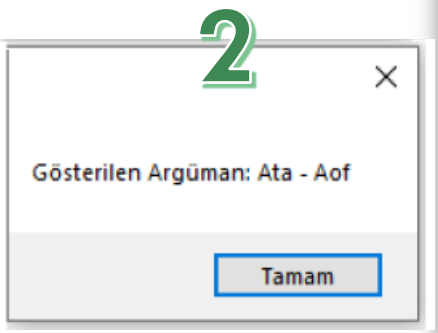
- **Kural 1:** Metot isimleri aynı olacak
- **Kural 2:** Parametre sayısı, tipi veya sırası farklı olacak



METOT AŞIRI YÜKLEME YÖNTEMLERİ

- Metot aşırı yükleme; parametre sayısını, parametre veri tipini ve parametre sırasını değiştirerek gerçekleştirilebilir.
- **Parametre Sayısını Değiştirme:** Metotlardaki parametre sayıları farklı olduğu için aynı isimde birden fazla metodun olması mümkündür.



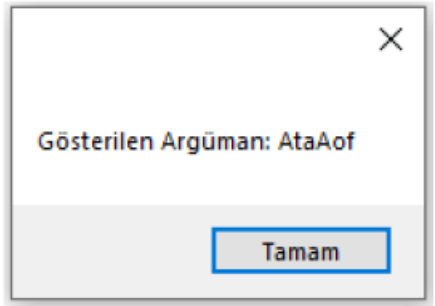


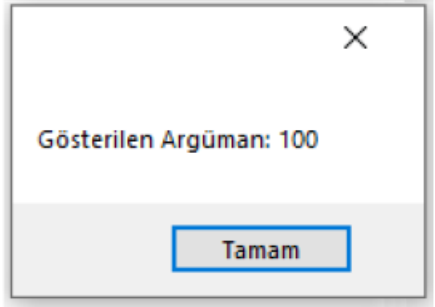

```
void Göster(string a)
{
   MessageBox.Show("Gösterilen Argüman: " + a);
}
void Göster(string a, string b)
{
   MessageBox.Show("Gösterilen Argüman: " + a + " - " + b);
}
private void Form1_Load(object sender, EventArgs e)
{
   Göster("AtaAof");
   Göster("Ata", "Aof");
}
```

METOT AŞIRI YÜKLEME YÖNTEMLERİ

- **Parametre Veri Tiplerini Değiştirerek:** Aynı sayıda parametre sayısına sahip ancak parametrelerinin veri türünün farklı olduğu metotların da aşırı yüklenmesi mümkündür.

```
void Göster(string a)
{
   MessageBox.Show("Gösterilen Argüman: " + a);
}
void Göster(int a)
{
   MessageBox.Show("Gösterilen Argüman: " + a.ToString());
}
private void Form1_Load(object sender, EventArgs e)
{
   Göster("AtaAof");
   Göster(100);
}
```

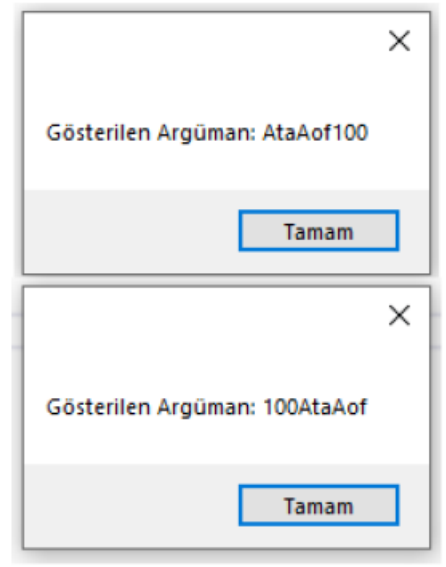




METOT AŞIRI YÜKLEME YÖNTEMLERİ

- **Parametrelerin Sırasını Değiştirerek:** Aynı sayıda parametre sayısına sahip ve parametrelerinin veri türünün aynı olduğu ancak parametre sırasının farklı olduğu metotların da aşırı yüklenmesi mümkündür.

```
void Göster(string a, int b)
{
   MessageBox.Show("Gösterilen Argüman: " + a + b.ToString());
}
void Göster(int a, string b)
{
   MessageBox.Show("Gösterilen Argüman: " + a.ToString() + b);
}
private void Form1_Load(object sender, EventArgs e)
{
   Göster("AtaAof",100);
   Göster(100, "AtaAof");
}
```



ÖZYİNELEMELİ (RECURSIVE) METOTLAR

- Özyineleme, bir işlevin kendini tekrar çağırarak çözüm üretme tekniğidir.
- Programlama ve matematikte önemli bir rol oynar; arama, sıralama algoritmaları ve veri yapılarında sıkça kullanılır.
- Özyinelemeli metotlar, karmaşık problemleri daha okunabilir ve kısa kodlarla çözmek için etkili bir yöntemdir; ancak bellek ve hız açısından dikkatli kullanılmalıdır.

ÖZYİNELEMELİ (RECURSIVE) METOTLAR

- **Temel Bileşenler**
  - **Temel Durum (Base Case):** İşlevin kendisini çağırmadığı durum; sonsuz döngüyü önler.
  - **Genel Durum (General Case):** Problemi daha küçük parçalara ayırarak çözme süreci.
  - **Yakınlaşma (Convergence):** Temel duruma ulaşma süreci.

ÖZYİNELEMELİ (RECURSIVE) METOTLAR

- **Avantajları**
  - Daha az kod ile yüksek işlevsellik.
  - Okunabilirlik artışı.
- **Dezavantajları**
  - Daha yavaş çalışma hızı.
  - Sonsuz döngü riski.
  - Bellek sorunları yaşanabilir.

ÖZYİNELEMELİ VE İTERATİF METOTLAR KARŞILAŞTIRMASI

- **Genel Yapı**
  - Temel durum ve genel durum arasında if-else yapısı.
  - Temel durum sağlanmadığında problem daha küçük parçalara bölünerek çözülür.

- **NOT: İteratif metot,** özyinelemeli olmayan ve genellikle döngüler kullanılarak oluşturulur.

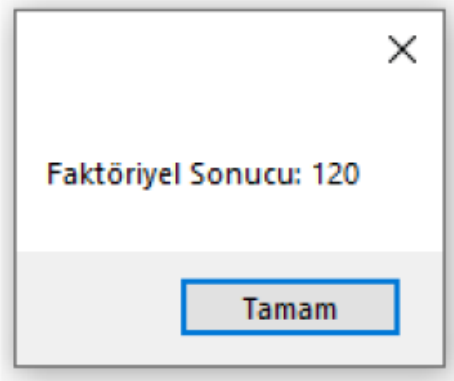
ÖZYİNELEMELİ VE İTERATİF METOTLAR KARŞILAŞTIRMASI

Özellik	Özyinelemeli Metot	İteratif Metot
Tanım	Kendini çağırır.	Tekrar eden talimat dizisi.
Başvuru	Metotlar için.	Döngüler için.
Sonlandırma	Temel durumla.	Yineleyici sonlandırma koşuluyla.
Kullanım	Küçük kod boyutu ve zaman karmaşık olmadığında.	Zaman karmaşıklığı dengelendiğinde.
Kod Boyutu	Daha küçük.	Daha büyük.
Zaman Karmaşıklığı	Yüksek (genellikle üstel).	Daha düşük (genellikle polinom).

ÖZYİNELEMELİ (RECURSIVE) METOTLAR

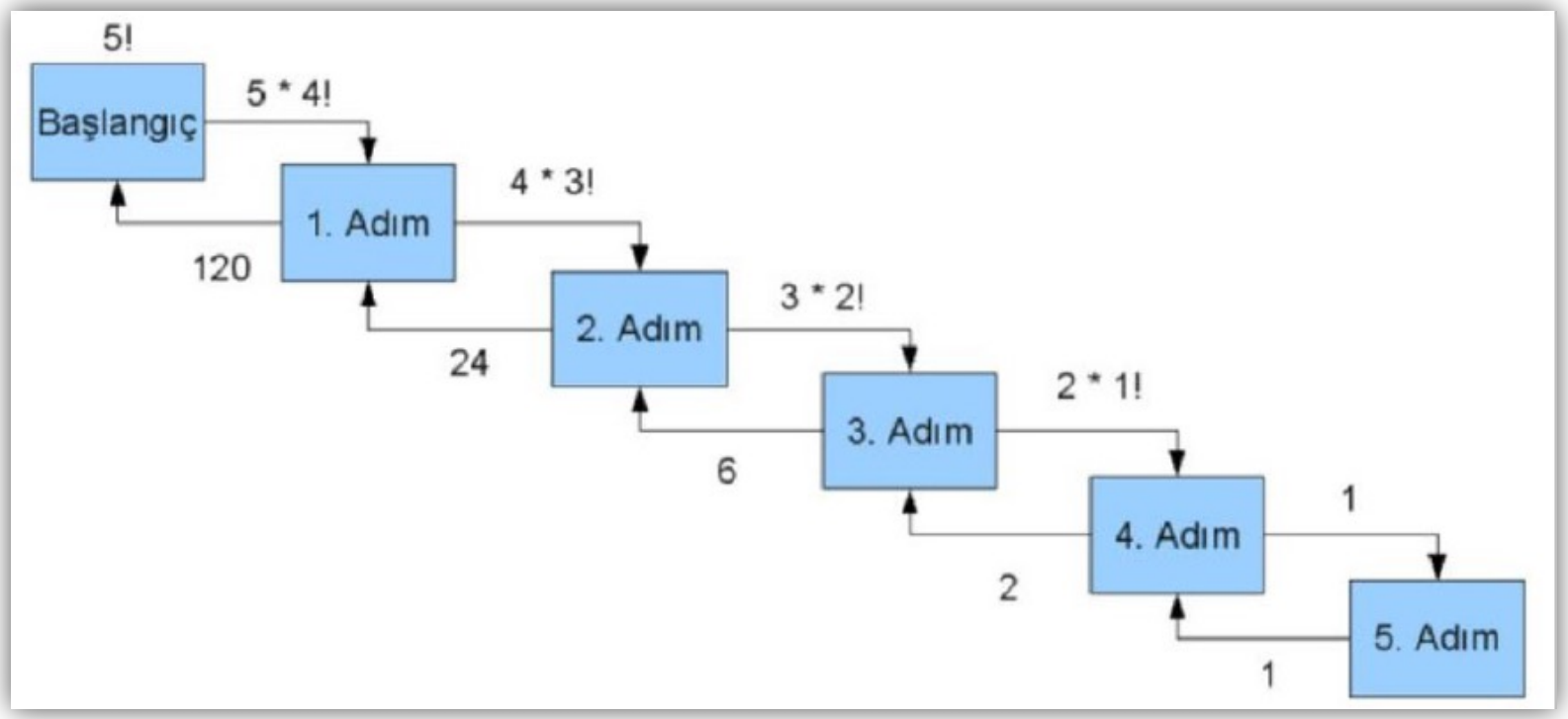
- **Faktöriyel hesaplayan örnek bir özyinelemeli (recursive) metot:**

```
int OzyinelemeFaktoriyelHesaplama(int m)
{
   if (m == 0 || m == 1)
   {
      return 1;
   }
   return m * OzyinelemeFaktoriyelHesaplama(m - 1);
   // m değişkeni, tekrar faktoriyel değişkeninden gelen ile çarpılır.
}
private void Form1_Load(object sender, EventArgs e)
{
   int sonuc = OzyinelemeFaktoriyelHesaplama(5);
   MessageBox.Show("Faktöriyel Sonucu: " + sonuc.ToString());
}
```



ÖZYİNELEMELİ (RECURSIVE) METOTLAR

- **5 in faktöriyel hesabını yapan özyinelemeli metodun çalışma mantığı**



ÖZYİNELEMELİ (RECURSIVE) METOTLAR

- **Aynı görevi yerine getiren iteratif metot;**

```
int IteratifFaktoriyelHesaplama(int m)
{
   int faktoriyel = 1;
   for (int i = 1; i <= m; i++)
   {
      faktoriyel *= i;
   }
   return faktoriyel;
}
private void Form1_Load(object sender, EventArgs e)
{
   int sonuc = IteratifFaktoriyelHesaplama(5);
   MessageBox.Show("Faktöriyel Sonucu: " + sonuc.ToString());
}
```

ÖZYİNELEMELİ (RECURSIVE) METOTLAR

- Fibonacci serisinin ilk elemanı 0 (sıfır), ikinci elemanı 1 'dir. Serinin diğer elemanlarının değeri, kendisinden önce gelen iki elemanın değeri toplanarak hesaplanır.

- Örnek fibonacci serisi elemanları

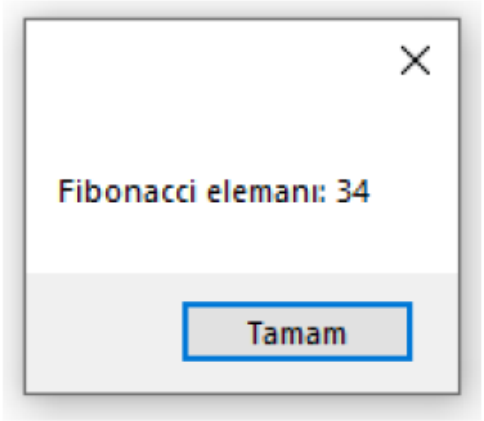
- {0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, ...}

ÖZYİNELEMELİ (RECURSIVE) METOTLAR

Fibonacci serisi elemanlarının hesabını yapan örnek bir özyinelemeli (recursive) metot:

```
int FibonacciHesaplama(int m)
{
   if (m == 0)
      return 0;
   else if(m == 1)
      return 1;
   else
      return FibonacciHesaplama(m - 1) + FibonacciHesaplama(m - 2);
}

private void Form1_Load(object sender, EventArgs e)
{
   int sonuc = FibonacciHesaplama(9);
   MessageBox.Show("Fibonacci elemanı: " + sonuc.ToString());
}
```



ÖZYİNELEMELİ (RECURSIVE) METOTLAR

Özyinelemeli (recursive) olarak üs alma görevini yerine getiren örnek metot:

```
int OzyinelemeUsAlma(int taban, int us)
{
   int sonuc;
   if (us == 0)
      sonuc = 1;
   else
      sonuc = taban * OzyinelemeUsAlma(taban, us - 1);
   return sonuc;
}

private void Form1_Load(object sender, EventArgs e)
{
   int sonuc = OzyinelemeUsAlma(5,4);
   MessageBox.Show("Recursive Us Alma: " + sonuc.ToString());
}
```

